

THE LEGACY OF ALBERTA

DE OFFICIËLE HINTGIDS

Deel 1 – milde duwtjes, geen antwoorden

een gids in twee delen
voor de zolder van grootmoeder Alberta

WELKOM, DOORZETTER!

Zit je vast op Alberta's zolder? Geen paniek. Dit is precies waarvoor deze gids bestaat. Hier krijg je duwtjes – kleine, milde duwtjes – die je weer op weg helpen zonder dat ze het antwoord verklappen. Zo hoort een echte gids te werken: jij lost het op, wij houden alleen de zaklamp vast.

Zo gebruik je deel 1. Zoek het level waar je vastzit. Bij elke puzzel staan twee of drie hints, van vaag naar iets minder vaag. Lees er ééntje. Ga terug naar het spel. Werkt het? Prachtig, sluit de gids. Werkt het nog niet? Lees de volgende hint. Blijf niet doorbladeren tot je per ongeluk te veel weet – dat bederft de pret.

Onthoud: in het spel zelf typ je gewoon `?` voor exact deze hints, in oplopende stappen. Deze gids herhaalt de eerste twee stappen op papier, zodat je ze rustig naast je toetsenbord kan leggen. De derde, meest verklappende stap laten we hier bewust weg – die zoek je in het spel, of, als het écht moet, in deel 2. En deel 2? Dat zit achter een zegel. Om een reden.

Nog iets: Alberta houdt bij hoeveel hints je vraagt. Niet om je te straffen – hints kosten je niets en blokkeren niets – maar om je op het einde een warme, droge terugblik te geven. Wie zonder hints geraakt, krijgt de knipoog van de meesterhand. Wie er een hoop nodig had, hoort: "We hebben dit samen gedaan." Allebei goed. Het spel draait, dat is wat telt.

DE ZOLDER: HOE JE RONDLOOPT EN FRAGMENTEN VINDT

Alberta's zolder is je thuisbasis. Je loopt er rond met de pijltjestoetsen, of je typt commando's zoals in een klassiek adventure: `kijk`, `ga oost`, `onderzoek notitieboek`, `open doos`, `ga zitten`. Typ `help` voor de volledige lijst, en `?` als je niet weet waar het volgende stukje zit.

De grote lijn is simpel en gaat altijd in dezelfde vijf stappen:

1. **Vind het fragment.** Elk level begint met een blad uit Alberta's beschadigde notitieboek. Het eerste ligt gewoon open in de westhoek. De latere bladen zitten in dichtgeplakte, gemerkte dozen, dieper in de zolder (de doorgang in het midden en de overloop boven aan de trap).
2. **Lees de spread.** Het notitieboek klap open: Alberta's schets, haar puzzelbrief, en de regel "Dit zou je moeten kunnen na week X van de cursus". Die weekregel zegt je of je er al aan toe bent.
3. **Ga aan de pc zitten.** Loop naar de werkhoeke in het oosten en typ `ga zitten`. Daar werk je aan haar Java-code.
4. **Los de puzzels op.** Meestal één editor-puzzel en twee terminal-puzzels per level. Alle drie af? Dan is dat hoofdstuk van Alberta's spel hersteld.
5. **Keer terug.** De pc sluit (`ESC`), je voortgang wordt bewaard, en je staat weer op de zolder, klaar voor het volgende fragment.

Vastgelopen op de zolder zelf? Bijna altijd is het antwoord: je hebt het volgende blad nog niet, of je zit in de verkeerde kamer. Typ `?` – Alberta wijst je naar de kamer waar het eerstvolgende fragment ligt. De grote doos in het midden met "BRONCODE" erop telt trouwens pas op het einde; laat die nog even dicht.

Meer dan dit zeggen we niet over de zolder. Het is een kleine, warme ruimte – je hebt ze zo in de vingers.

LEVEL 1 – DE BLAUWDruk EN DE DOOS

Scharnier: klasse en instantie, velden, constructor, `this`. Speelbaar na week 1.

Het startpunt, en meteen het fundament van alles. Je herstelt Alberta's **Voorwerp** en schrijft haar **Geitje** uit wat er van de notities rest. Het kernbeeld: een klasse is een **blauwdruk** (het plan dat je één keer tekent); een object is de **doos** die je ernaar bouwt. De constructor vult de velden van zo'n verse doos.

Puzzel 1 – herstel de constructor van Voorwerp. De waterschade vrat een verwijzing naar de doos zelf op.

- *Hint 1:* Denk aan de blauwdruk en de doos. De constructor vult de velden van een verse doos. Wat gaat waarheen?
- *Hint 2:* Kijk naar de toewijzingen in de constructor. Bij één veld ontbreekt de verwijzing naar de doos zelf, of ze staat net omgekeerd.

Puzzel 2 – schrijf Geitje van nul. Alberta somt in de kantlijn op wat een geitje heeft; jij tikt de klasse uit.

- *Hint 1:* Een klasse is een blauwdruk: eerst de velden (wat een geitje heeft), dan de constructor die de verse doos vult, dan de getters.
- *Hint 2:* Je mist nog een privaat veld, een toewijzing in de constructor, of een getter. Vergelijk met wat de notitie opsomt (en let op: de gered-vlag begint gewoon op false).

Puzzel 3 – verklaar in één zin: klasse versus instantie. Dit beoordeel je daarna zelf tegen Alberta's model.

- *Hint 1:* Blauwdruk en doos: het ene is het plan, het andere het ding dat je ermee bouwt.
- *Hint 2:* Welk woord hoort bij het plan (je tekent het één keer), en welk bij het ding (je maakt er vele van)?

LEVEL 2 – TRECHTERS ERIN, GOOT ERUIT

Scharnier: signaturen (return vs. `void`), en het verschil tussen attribuut, parameter en lokale variabele. Speelbaar na week 2.

Nu draait alles om methode-koppen. Het beeld: een methode is een machine. Trechters erin (de parameters), een goot eruit (het returntype) – of niets eruit (`void`). En drie soorten dozen om waarden in te bewaren: attribuut, parameter, lokale variabele.

Puzzel 1 – herstel de signaturen van Speler. De koppen zijn tot pap doorgelopen: eentje geeft iets terug maar zegt `void`, of een trechter is verdwenen.

- *Hint 1:* Trechters erin, goot eruit – of niets eruit. Kijk per methode: wat gaat erin, wat komt eruit?
- *Hint 2:* Kijk naar de koppen. Eén geeft iets terug maar zegt `void`; bij een andere is de trechter (de parameter) verdwenen. Een getter geeft iets terug, een setter neemt een waarde aan.

Puzzel 2 – Parsons: orden de zoekmethode. Sleep de stroken in de juiste volgorde. Let op: één strook hoort er níét bij.

- *Hint 1:* De methode begint met haar kop, werkt van boven naar onder, en sluit met een accolade.
- *Hint 2:* Eén strook geeft de verkeerde doos terug – de parameter in plaats van het gevonden object. Die afleider laat je liggen.

Puzzel 3 – trace: een parameter schaduwet een attribuut. Voorspel de twee getallen die verschijnen.

- *Hint 1:* Drie dozen: attribuut, parameter, lokale variabele. Welke doos bedoelt de naam hier?
- *Hint 2:* Zonder `this` pakt de code de dichtstbijzijnde doos: de parameter. Met `this` de doos van het object (het attribuut).

LEVEL 3 – DE KNIKKERBAAN

Scharnier: voorwaarden – validatie, cascade, `&&` / `||` / `!`. Speelbaar na week 2.

De kern van toets 1. Het beeld: een knikkerbaan. De validatie klemt de knikker tussen twee randen; de cascade splitst de baan; `&&` / `||` / `!` sturen welke kant hij op rolt.

Puzzel 1 – herstel de klem in `setLevenspunten`. De waarde moet tussen twee randen blijven: nooit onder 0, nooit boven het maximum.

- *Hint 1:* De knikkerbaan: de waarde moet tussen twee randen blijven, nooit onder de ene, nooit boven de andere.
- *Hint 2:* Kijk naar de twee `if`-controles. Eén rand staat de verkeerde kant op, of een rand ontbreekt helemaal.

Puzzel 2 – vind de fout: `&&` versus `||`. Een poort mag alleen open als de wolf verslagen is ÉN je de sleutel hebt. Toch klopt er iets niet.

- *Hint 1:* De baan stuurt met `&&` (en) / `||` (of): welke van de twee laat de knikker door als béide sporen moeten kloppen?
- *Hint 2:* Lees de opgave: de poort mag maar open als álle voorwaarden waar zijn. Kijk naar de operator in de conditie op regel 1.

Puzzel 3 – trace: de validatie-cascade op een randwaarde. Welk woord verschijnt? Let op de randen.

- *Hint 1:* De cascade splitst de baan: de knikker rolt in de eerste tak die klopt, en dan niet meer verder.
- *Hint 2:* Let op de randen: `<= 0` pakt ook net 0, en `< 10` pakt 10 net niet. Waar valt jouw waarde? Loop de takken van boven naar onder.

LEVEL 4 – TWEE PIJLEN, ÉÉN DOOS

Scharnier: referenties (aliasing) en `null`. Speelbaar na week 3.

Dit scharnier draagt alle latere lessen. Het beeld: twee variabelen kunnen naar dezelfde doos wijzen – twee pijlen, één doos. Verander je de doos via de ene pijl, dan ziet de andere het ook. En `null` is een pijl die naar geen enkele doos wijst.

Puzzel 1 – herstel de buur-bedrading (`verbindKamers`). Elke verbinding tussen kamers loopt twee kanten op. Wat de ene kant legt, moet de andere kant terugleggen.

- *Hint 1:* Twee pijlen, één doos: elke verbinding loopt twee kanten op. Leg je de noord-buur, leg dan meteen de zuid-buur terug.
- *Hint 2:* Kijk naar de takken van de cascade. Bij één richting is de setter verkeerd, zodat de terugweg nooit meer wordt gelegd. Alle vier de richtingen horen in één `if` / `else if`-cascade.

Puzzel 2 – trace: aliasing. Twee variabelen wijzen naar dezelfde kamer; je zet de buur via de ene en leest hem via de andere.

- *Hint 1:* Twee pijlen, één doos: `eerste` en `tweede` wijzen naar dezelfde kamer.
- *Hint 2:* Wat je via `tweede` zet, staat ook in `eerste` – het blijft dezelfde doos. Dus is `eerste.getNoord()` precies wat je met `tweede` gezet hebt.

Puzzel 3 – verklaar in één zin: wat betekent `null` hier?

- *Hint 1:* Twee pijlen, één doos – en `null` is een pijl die naar géén doos wijst.
- *Hint 2:* Denk aan een kamer zonder uitgang in die richting: waar wijst de buur-referentie dan heen?

LEVEL 5 – DE PATROONKAART

Scharnier: de lus-romp + patroonkeuze (tellen, totaliseren, opbouwen, filteren, het uiterste). Speelbaar na week 4.

De kern van toets 2. Elke lus volgt een patroonkaart. Kies eerst de kaart, dan schrijf je de romp bijna vanzelf.

Puzzel 1 – schrijf twee lus-methoden. `telWapens` telt (een teller die ophoogt); `sterksteVoorwerp` zoekt het uiterste (onthoud de beste tot nog toe).

- *Hint 1:* Welke patroonkaart? `telWapens` is de tel-kaart; `sterksteVoorwerp` is de uiterste-kaart.
- *Hint 2:* Begin `telWapens` met een teller op 0 en hoog op bij een treffer. Begin `sterksteVoorwerp` met `null` en vervang zodra je iets sterkers ziet. Beide lopen met een `for`-lus over de inventaris.

Puzzel 2 – Parsons: de string-builder. Bouw een kamer-regel die met elke ronde langer wordt. Eén strook hoort er niet bij.

- *Hint 1:* De opbouw-kaart: je maakt een regel die met elke ronde langer wordt.
- *Hint 2:* Eén strook overschrijft de regel in plaats van eraan toe te voegen – dat is de klassieke opbouw-fout, en die hoort er niet bij.

Puzzel 3 – welke patroonkaart? Een lus uit het schade-overzicht. Kies uit tellen, totaliseren, opbouwen of het uiterste.

- *Hint 1:* Kijk wat de lus met elk element doet: telt ze er één bij, of telt ze de waarde zélf op?
- *Hint 2:* Bij `som = som + schadeLog[i]` groeit `som` met de waarde, niet met één per element. Eén per element is tellen; de waarde optellen is iets anders.

LEVEL 6 – DE PLANKENBRUG BOVEN HET RAVIJN

Scharnier: index en off-by-one; welke lus kies ik. Speelbaar na week 5.

Toets 2 en de eindtoets, allebei. Het beeld: een lijst is een plankenbrug boven een ravijn. De eerste plank is nummer 0; de laatste is `size()` min één. Eén plank te ver en je ligt in het water.

Puzzel 1 – herstel de off-by-one in verwijderVoorwerp. Een lus loopt één plank te ver, of het is de verkeerde soort lus.

- *Hint 1:* De plankenbrug: de eerste plank is 0, de laatste is `size()` min één. Eén plank te ver en je ligt in het water.
- *Hint 2:* Kijk naar de lusgrens, of naar de soort lus: klopt `<` versus `<=`, en is het wel de lus die hier past?

Puzzel 2 – trace: de laatste afgedrukte index. Voor een lijst met N voorwerpen: welk getal wordt als laatste afgedrukt?

- *Hint 1:* Tel de planken vanaf 0, niet vanaf 1.
- *Hint 2:* De lus stopt zodra `i` niet meer kleiner is dan `size()`. De laatste `i` die nog gedrukt wordt, is `size()` min één.

Puzzel 3 – vind de fout: de rondelus. Een gevechtslus hoort precies vijf rondes te spelen (0 tot en met 4). Toch klopt er iets niet.

- *Hint 1:* De plankenbrug: hoe ver loopt de lus, en is dat één plank te ver?
- *Hint 2:* Kijk naar de grens van de `for`-lus op regel 1, niet naar de romp. Vijf rondes vraagt `< 5`, geen `<= 5` – dat laatste speelt er zes.

LEVEL 7 – DE SPEURTOCHT EN DE DUBBELE PIJL

Scharnier: zoeken + de dubbele pijl (een ketting van getters). Speelbaar na week 6.

Het laatste hoofdstuk, en de kern van de eindtoets. Een zoeklus is een speurtocht: hij geeft het gevonden object terug, of `null` als de tocht doodloopt. En soms volg je twee pijlen na elkaar – een ketting van getters, die je null-veilig moet houden.

Puzzel 1 – schrijf de zoeklus (zoekGeitje). Loop de lijst af en geef het geitje met de gezochte naam terug, of `null` als het er niet is.

- *Hint 1:* De speurtocht: een zoeklus loopt de lijst af, geeft het gevonden object terug, of `null` als de tocht doodloopt.
- *Hint 2:* Loop met een `for`-lus over de geitjes, vergelijk elke naam, en geef bij een treffer meteen het geitje terug. Kom je aan het einde zonder treffer, dan `null`.

Puzzel 2 – herstel de null-veilige keten. `geitje.getSchuilplaats().getKamer()` – maar een geitje zonder schuilplaats heeft geen kamer om naar te wijzen.

- *Hint 1:* De dubbele pijl: soms wijst de eerste pijl naar `null`, en dan is er geen tweede pijl om te volgen.
- *Hint 2:* Er ontbreekt een null-controle vóór de tweede pijl, of de keten mist een schakel. Haal eerst de schuilplaats op en controleer of ze `null` is, vóór je verder gaat.

Puzzel 3 – trace: de geketende getter, met het null-geval. Twee geitjes, eentje met en eentje zonder schuilplaats. Wat verschijnt er?

- *Hint 1:* De speurtocht met de dubbele pijl: soms wijst de eerste pijl naar `null`.

- *Hint 2:* Controleer eerst of de schuilplaats **null** is. Is ze **null**, dan stopt de keten op “nog niet gevonden”. Anders volg je de twee pijlen naar de kamernaam.

HET EINDSPEL: SEVEN LITTLE GOATS

Level 7 af? Dan boot de pc Alberta's spel als speelbare simulatie in de terminal, en speel je eindelijk *Seven Little Goats* uit. Je bent Roodkapje. Je volgt het spoor van de jonge wolf van het geitenhuisje tot de rivieroever, en rekent daar met hem af.

We houden het hier bewust luchtig – geen kaart, geen recept, geen exacte commando's. Dat is de fijne ontdekking, die willen we je niet afpakken. Wel drie eerlijke richtwijzers:

- **Verken en verzamel.** Loop de kamers af, **pak** wat je tegenkomt, **bekijk** je voorwerpen en **praat** met wie er is. Sommige voorwerpen genezen je, eentje telt mee bij elke aanval, en eentje uit grootmoeders huisje opent meer dan stof. Een **mandje** heb je nodig om iets te kunnen dragen, en een raaf in het bos ruilt graag.
- **Het wolfgevecht telt.** Vijf rondes, een vast patroon, geen toeval – wie rékent, wint. Zorg dat je scherp bent voor je **vecht** typt: het juiste gereedschap op zak maakt het verschil tussen net winnen en kopje-onder gaan. De jachthond bij de molen is optioneel; een omweg, geen verplichting.
- **Sparen of afmaken is een échte keuze.** Als de wolf op het einde smeekt, is wat je dan kiest geen detail – het bepaalt mee welk van de vier eindes je krijgt. Er is een best einde dat twee dingen tegelijk vraagt, een kille variant, een genadige, en natuurlijk een game over. Welke je “hoort” te spelen, beslis jij. Ze zijn geen van alle fout.

Vast in de sim? Typ **?** voor een hint die bij je huidige plek past, of **opties** voor een compleet overzicht van wat hier kan – dat laatste is een cheatcode, en dat weet het spel. Gebruik hem met mate.

En de exacte weg naar elk van de vier eindes, met alle commando's op een rij? Die staat in deel 2. Achter het zegel. Je weet wat je te doen staat – of net niet te doen.

Veel succes, doorzetter. Alberta kijkt mee.